

QUBE: Quasi-Cyclic with Unbalanced & Multi-Block Errors

Contents

1	Specification of the QUBE Algorithm.....	3
1.1	Overview	3
1.2	Preliminaries and Notations	3
1.3	The Public Key Encryption Scheme QUBE.PKE	6
1.4	The Key Encapsulation Mechanism Scheme QUBE.KEM	7
1.5	Parameters and Settings	7
1.6	Sizes of Keys and Ciphertexts	8
2	Design Rationale.....	9
2.1	Code-based Hard Computational Problems	9
2.2	QCSD Problems underlying QUBE	13
2.3	Analysis of Decryption Failure Rate	14
3	Security Analysis	16
3.1	Reduction-based Security.....	16
3.2	Practical Security	20
3.2.1	ISD Algorithms for QCSD Problems.....	20
3.2.2	Quantum ISD Algorithms for QCSD Problems.....	22
3.3	Other Attacks	23
3.4	Our Attacks on HQC	23
4	Performance Evaluations.....	24
4.1	Implementation Notes.....	24
4.2	Metrics and Results	25
5	Feature Statements.....	25
	References	26

1 Specification of the QUBE Algorithm

1.1 Overview

In constructing QUBE, we first propose an IND-CPA-secure public-key encryption scheme QUBE.PKE, whose security is based on hardness of the quasi-cyclic syndrome decoding (QCSD) problem with unbalanced blockwise errors. Then by applying Fujisaki-Okamoto (FO) transform [9] to QUBE.PKE, we obtain an IND-CCA2-secure key encapsulation mechanism (KEM), denoted as QUBE.KEM or plainly QUBE.

QUBE generalizes the framework of the HQC algorithm [1], which has similar construction and security analysis, with a multi-block technique and unbalanced blockwise errors. Compared to HQC, the main advantage of QUBE is that it significantly reduces the size of ciphertexts by roughly 25%–30% across different security levels; moreover, the total size of public key plus ciphertext also gets smaller than that of HQC by roughly 1.4%–5.6%. We remark that the size of ciphertexts is especially critical when using KEM in static-ephemeral key agreement or KEM-based authentication like KEMTLS [19] with a static certificate; in normal ephemeral key agreements like in TLS, IPsec/IKE, or SSH, the total size of public key plus ciphertext often dominates the communication cost in post-quantum settings.

We provide the parameter sets of QUBE targeting 128, 256, 384 and 512 bits of classical security levels, which also stay securely above 80, 128, 192, and 256 bits of quantum security levels, respectively.

In the remaining of this section, we mainly focus on definitions and specifications for QUBE. The QUBE algorithm is further analyzed in later sections: we analyze the security foundations and decryption failure rates in Section 2, present formal security proofs and best-known attacks in Section 3, metric the performance in Section 4, and summarize the features and uniqueness of QUBE in Section 5.

1.2 Preliminaries and Notations

In this section, we provide some notations and definitions necessary for describing our scheme QUBE.

Algebra. Vectors and matrices are denoted by bold lowercase and uppercase letters, respectively. Unless otherwise specified, all vectors are assumed to be *row* vectors. We denote by $\mathbf{x}^\top$ the transposed column vector for the (row) vector $\mathbf{x}$.

We write $\mathbb{F}_2$ for the binary field, and $\mathbb{F}_2^r$ for the r -dimensional vector space over $\mathbb{F}_2$. We mainly consider the polynomial ring $\mathcal{R} = \mathbb{F}_2[X]/(X^r - 1)$ in our scheme; the concrete choices of r will be specified later. We also view every polynomial in the ring $\mathcal{R}$ as a vector in $\mathbb{F}_2^r$, i.e., a binary vector of length r , which lists the coefficients of the polynomial from lowest- to highest-degree terms; as a result, we also denote polynomials in $\mathcal{R}$ by bold lowercase letters, and furthermore denote vectors of polynomials by bold uppercase letters.

Let $\text{wt}(\mathbf{v})$ denote the Hamming weight of a vector $\mathbf{v}$; since we also regard polynomials in $\mathcal{R}$ as vectors, the notation $\text{wt}(\mathbf{v})$ naturally extends to polynomial $\mathbf{v}$. Given two vectors $\mathbf{n} = (n_1, \dots, n_\ell)$, $\mathbf{w} = (w_1, \dots, w_\ell) \in \mathbb{Z}_{\geq 0}^\ell$, we define set

$$\mathcal{S}_{\mathbf{w}}^{\mathbf{n}} := \{\mathbf{v} = (\mathbf{v}_1, \dots, \mathbf{v}_\ell) \in \mathbb{F}_2^{\ell r} \mid \mathbf{v}_i \in \mathbb{F}_2^{n_i}, \text{wt}(\mathbf{v}_i) = w_i \text{ for } 1 \leq i \leq \ell\}.$$

When $n_i = r$ are all equal, we simplify the notation as $\mathcal{S}_{\mathbf{w}}^r$. Again, since we regard polynomials in $\mathcal{R}$ as vectors, we also regard $\mathcal{S}_{\mathbf{w}}^r \subseteq \mathcal{R}^\ell$.

For sampling a random variable x uniformly at random from a finite set S , we write $x \xleftarrow{\$} S$. For example, with $(\mathbf{x}_1, \mathbf{x}_2, \mathbf{y}) \xleftarrow{\$} \mathcal{S}_{(w_x, w_x, w_y)}^r$ (used in key generation process), we mean the vector of polynomials $(\mathbf{x}_1, \mathbf{x}_2, \mathbf{y})$ is sampled uniformly at random from the set $\mathcal{S}_{(w_x, w_x, w_y)}^r$, meaning $\mathbf{x}_i$ and $\mathbf{y}$ have Hamming weights w_x and w_y , respectively.

A binary linear code $\mathcal{C}$ of length n and dimension k is defined as a subspace of $\mathbb{F}_2^n$ with dimension k ; typically this is called a binary $[n, k]$ -linear code. If the rows of a matrix $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ form a basis of the binary code $\mathcal{C}$, then the matrix $\mathbf{G}$ is called a generator matrix of $\mathcal{C}$. Moreover, a matrix $\mathbf{H} \in \mathbb{F}_2^{(n-k) \times n}$ is called a parity-check matrix of $\mathcal{C}$ if its (right-)kernel space gives exactly $\mathcal{C}$. For vector $\mathbf{x} \in \mathbb{F}_2^n$, its syndrome vector $\mathbf{s}$ in a code with parity-check matrix $\mathbf{H}$ is defined by $\mathbf{s} = \mathbf{x}\mathbf{H}^\top$; we have that $\mathbf{x} \in \mathcal{C}$ if and only if it has zero syndrome $\mathbf{x}\mathbf{H}^\top = \mathbf{0}$.

The minimum distance of a linear code $\mathcal{C}$ is defined by $\min\{\text{wt}(\mathbf{c}) : \mathbf{c} \in \mathcal{C}, \mathbf{c} \neq \mathbf{0}\}$. If a binary $[n, k]$ -linear code $\mathcal{C}$ has minimum distance d , then it is further called a binary $[n, k, d]$ -linear code.

Let $\mathbf{v} = (v_0, \dots, v_{r-1}) \in \mathbb{F}_2^r$, which to remind also represents the polynomial $v(x) = \sum_{i=0}^{r-1} v_i x^i \in \mathcal{R}$. The circulant matrix induced by $\mathbf{v}$ is defined as

$$\mathbf{rot}(\mathbf{v}) = \begin{pmatrix} v_0 & v_{r-1} & \cdots & v_1 \\ v_1 & v_0 & \cdots & v_2 \\ \vdots & \vdots & \ddots & \vdots \\ v_{r-1} & v_{r-2} & \cdots & v_0 \end{pmatrix} \in \mathbb{F}_2^{r \times r}.$$

It can be verified that for two polynomials $\mathbf{u}, \mathbf{v} \in \mathcal{R}$, which are also viewed as coefficient vectors, their polynomial product is linked to a vector-matrix product as $\mathbf{u} \cdot_{\text{poly}} \mathbf{v} = \mathbf{u} \cdot_{\text{mat}} \mathbf{rot}(\mathbf{v})^\top$.

For positive integers s and r , an $[sr, k]$ -linear code $\mathcal{C}$ is called a quasi-cyclic code of index s if, for any codeword $\mathbf{c} \in \mathcal{C}$, where we write $\mathbf{c} = (\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_s)$ and $\mathbf{c}_i \in \mathbb{F}_2^r$, and for any cyclic shift operation shift_ℓ for $\ell = 1, 2, \dots, r-1$, the vector $\mathbf{c}' = (\text{shift}_\ell(\mathbf{c}_1), \text{shift}_\ell(\mathbf{c}_2), \dots, \text{shift}_\ell(\mathbf{c}_s))$ (i.e., to simultaneously apply the cyclic shift to every block) is also a codeword in $\mathcal{C}$. For positive integers $s \geq t$ and r , an $[sr, tr]$ -linear code is called a systematic quasi-cyclic code of index (s, t) if it has the following $(s-t)r \times sr$ parity-check matrix:

$$\mathbf{H} = \begin{pmatrix} \mathbf{I}_r & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{H}_{1,1} & \cdots & \mathbf{H}_{1,t} \\ \mathbf{0} & \mathbf{I}_r & \ddots & \vdots & \mathbf{H}_{2,1} & \cdots & \mathbf{H}_{2,t} \\ \vdots & \ddots & \ddots & \mathbf{0} & \vdots & \cdots & \vdots \\ \mathbf{0} & \cdots & \mathbf{0} & \mathbf{I}_r & \mathbf{H}_{s-t,1} & \cdots & \mathbf{H}_{s-t,t} \end{pmatrix}, \quad (1)$$

where each $\mathbf{H}_{i,j} = \mathbf{rot}(\mathbf{h}_{i,j})$ is the $r \times r$ circulant matrix induced by an underlying polynomial $\mathbf{h}_{i,j} \in \mathcal{R}$. We note that the special case $t = 1$ is used in HQC [1].

Error-correcting codes. Same as in HQC [1], in QUBE we use shortened Reed-Solomon codes, duplicated Reed-Muller codes, and concatenated codes of them.

For Reed-Solomon codes, we follow exactly the definitions and code encoding/decoding algorithms given in [10, Section 3.4.2]. In particular, we consider Reed-Solomon codes over field $\mathbb{F}_{2^8}$ with a full evaluation set $\mathbb{F}_{2^8}^*$, which are $[\hat{n}_1 = 255, \hat{k}_1, d_1 := \hat{n}_1 - \hat{k}_1 + 1]$ -codes, and the versions shortened by length S as $[n_1 = \hat{n}_1 - S, k_1 = \hat{k}_1 - S, d_1]$ -codes.

For Reed-Muller codes, we also follow exactly the definitions and code encoding/decoding algorithms given in [10, Section 3.4.3]. In particular, we consider the $(7, 1)$ -Reed-Muller code, which is a $[\hat{n}_2 = 2^7, k_2 = 8, \hat{d}_2 = 2^6]$ -code, and the versions duplicated by factor D as $[n_2 = D \cdot \hat{n}_2, k_2, d_2 = D \cdot \hat{d}_2]$ -codes.

Given a $[n_1, k_1, d_1]$ -Reed-Solomon code over $\mathbb{F}_{2^8}$ and a $[n_2, k_2 = 8, d_2]$ -Reed-Muller code, their concatenated code, as defined in [10, Section 3.4.1], is a $[N = n_1 n_2, K = k_1 k_2, D \geq d_1 d_2]$ -code with message space $\mathbb{F}_{2^8}^{k_1}$ and code space $\mathbb{F}_2^N$.

Formatting and symmetric cryptographic subroutines. We write $\mathbb{B} = \mathbb{Z}_{2^8}$ for the set represented by a byte. For a vector/polynomial $\mathbf{v} \in \mathbb{F}_2^\ell$, we define $v = \text{Pack}(\mathbf{v}) \in \mathbb{B}^{\lceil \ell/8 \rceil}$ to be the byte string that packs the vector sequentially, from least significant bit to most significant bit inside a byte; e.g., if $\mathbf{v} = (v_0, v_1, \dots, v_{\ell-1})$ (and $\ell \geq 8$), then the first byte of v is $\sum_{i=0}^7 v_i \cdot 2^i$. Conversely, for a byte string $v \in \mathbb{B}^L$, we define $\mathbf{v} = \text{Unpack}(v, \ell) \in \{0, 1\}^\ell$ to be the inverse of Pack that unpacks each byte back to bits (from least significant bit to most significant bit); here ℓ specifies a truncated length and must satisfy $8(L - 1) < \ell \leq 8L$, i.e., $L = \lceil \ell/8 \rceil$. We also extend them to vectorized versions $\text{Pack}(\mathbf{V})$ and $\text{Unpack}(v, (\ell_1, \ell_2, \dots))$ that respectively pack from and unpack to *vector* of vectors/polynomials, by naturally concatenating or slicing the entry vectors/polynomials one by one.

We denote the security parameter by λ . Recall that in QUBE, we consider $\lambda \in \{128, 192, 256, 384, 512\}$, which correspond to the classical security levels targeted. We take an extendable output function (XOF) $\text{XOF}(\rho)$ which is keyed by a seed ρ (usually of λ bits) and produces variable-length pseudorandom output. Now we specify how to sample vectors from the seed. We first define $\text{GenUniformVector}(\rho, \ell)$ for sampling uniform vectors: it calls $\text{XOF}(\rho)$ for ℓ bits, and directly list them as a vector in $\mathbb{F}_2^\ell$. Moreover, we define $\text{GenShortVector}(\rho, \ell, w)$ for sampling vectors with fixed Hamming weight w out of length ℓ uniformly at random, i.e., sampling $\mathbf{v} \xleftarrow{\$} \mathcal{S}_w^\ell$:

- it calls $\text{XOF}(\rho)$ for variable-length output in an iterative way;
- it maintains a vector $\mathbf{v} = (v_0, v_1, \dots, v_{\ell-1})$, initialized to all zeros, and a counter c initialized to zero;
- in each iteration, it asks for $\lceil \log_2(\ell) \rceil$ bits and view it as the binary expansion (from least significant bit to most significant bit) of an integer i ;
- if $i \geq \ell$, or if $v_i = 1$, then this iteration is skipped;

- otherwise it updates the vector entry $v_i \leftarrow 1$ and update $c \leftarrow c + 1$;
- when $c = w$ is reached, then $\mathbf{v} \in \mathbb{F}_2^\ell$ is returned; otherwise it continues the next iteration.

Furthermore, we allow vectorized versions $\text{GenUniformVector}(\rho, (\ell_1, \ell_2, \dots))$ and $\text{GenUniformVector}(\rho, \ell, (w_1, w_2, \dots))$ that generate *vector* of vectors/polynomials, by generating the entry vectors/polynomials one by one while reusing the same $\text{XOF}(\rho)$ call.

Besides XOF , QUBE also makes use of a PRG $\text{PRG} : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$ and hash functions $\mathbf{H} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, $\mathbf{J} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, and $\mathbf{G} : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$. Their instantiations will be introduced in Section 1.5.

For convenience, $\text{trunc}(\mathbf{x}, \ell)$ denotes the truncation of a binary vector to ℓ bits, and $a \parallel b$ denotes the concatenation of two *byte* strings a, b .

Polynomial multiplication subroutines. When multiplying two polynomials in $\mathcal{R}$, we utilize nontrivial polynomial multiplication algorithms to speed up the computation. In particular, we consider the Karatsuba algorithm and the Toom-Cook algorithm. These are the same choices as [10, Section 3.3], and we refer to it for the specific descriptions of the polynomial multiplication algorithms.

In particular, the Karatsuba algorithm (denoted K2) is a recursive algorithm that splits the two input polynomials into halves, and carry out 3 recursive sub-multiplications to accomplish the overall multiplication. Hence it has time complexity $O(d^{\log_2 3})$ for degree- d multiplication.

The Toom-Cook algorithm (denoted TC3) is a recursive algorithm that splits the two input polynomials into three equal, and carry out 5 recursive sub-multiplications to accomplish the overall multiplication. Hence it has time complexity $O(d^{\log_3 5})$ for degree- d multiplication. It can be observed that both Karatsuba and Toom-Cook has lower complexity than the trivial textbook $O(d^2)$ polynomial multiplication.

We note that both Karatsuba and Toom-Cook only need generic sub-polynomial multiplication when recursing. Therefore, it is possible to hybrid the algorithm. E.g., we write TC3-K2 for an algorithm that apply the Toom-Cook algorithm at the outermost layer, and calls the Karatsuba algorithm when recursing to multiply sub-polynomials (and still Karatsuba if further recursing).

We require a perfect split of the polynomials inside both Karatsuba and Toom-Cook. This means, e.g., K2 only works for polynomial degrees of the form 2^k , and TC3-K2 only works for $3 \cdot 2^k$, etc. If the target polynomial multiplication does not have the right form of degree, we pad the degree to match the form, run the polynomial multiplication algorithm, and truncate the degree of the product polynomial back.

1.3 The Public Key Encryption Scheme QUBE.PKE

In this section we present our public key encryption scheme QUBE.PKE, as shown in Figures 1 to 3. The scheme assumes the following global parameters:

QUBE.PKE.KeyGen(ρ) $\rightarrow$ ($\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}$):

Input: randomness $\rho \in \mathbb{B}^{\lambda/8}$.

Output: public key $\text{pk}_{\text{PKE}} \in \mathbb{B}^{\lambda/8+2 \cdot \lceil r/8 \rceil}$, private key $\text{sk}_{\text{PKE}} \in \mathbb{B}^{\lambda/8}$.

1. $(\rho_0, \rho_1) = \text{PRG}(\rho)$
2. $\mathbf{H} = (\mathbf{h}_1, \mathbf{h}_2) = \text{GenUniformVector}(\rho_0, (r, r)) \in \mathcal{R}^2$
3. $(\mathbf{y}, \mathbf{x}_1, \mathbf{x}_2) = \text{GenShortVector}(\rho_1, r, (w_y, w_x, w_x)) \in \mathcal{S}_{(w_y, w_x, w_x)}^r$, and write $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2) \in \mathcal{R}^2$
4. $\mathbf{S} = \mathbf{X} + \mathbf{y} \cdot \mathbf{H}$, i.e., $\mathbf{S} = (\mathbf{s}_1, \mathbf{s}_2) = (\mathbf{x}_1 + \mathbf{y} \cdot \mathbf{h}_1, \mathbf{x}_2 + \mathbf{y} \cdot \mathbf{h}_2)$
5. $\text{pk}_{\text{PKE}} = \rho_0 \parallel \text{Pack}(\mathbf{S})$
6. $\text{sk}_{\text{PKE}} = \rho_1$
7. return $\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}$

Fig. 1: The key generation of QUBE.PKE

security parameter λ , degree r of the polynomial ring, linear code $\mathcal{C}$ with dimension λ and length L , and Hamming weights w_x, w_y, w_f, w_g, w_e . We note that λ, L are always ensured to be multiple of 8.

The scheme QUBE.PKE is IND-CPA secure under the assumption that both the so-called decisional (3, 1)-QCS and (4, 2)-QCS problems are hard to solve; see the detailed definitions in Section 2.1 and the security proofs in Section 3.1.

1.4 The Key Encapsulation Mechanism Scheme QUBE.KEM

The KEM version QUBE.KEM is obtained by applying the FO_m^χ transform from [13] (with salt) to QUBE.PKE; its detailed procedures are shown in Figures 4 to 6. It assumes the same set of global parameters as QUBE.PKE.

1.5 Parameters and Settings

We propose four parameter sets of QUBE tailored to security levels of 128, 256, 384, and 512 bits, and denote them as QUBE-128, QUBE-256, QUBE-384, and QUBE-512, respectively; we also include a parameter set for security level 192 for reference and better comparison to the HQC algorithm. All parameters are chosen so that the bit complexity of the best known attack exceeds the targeted level with certain security margin. More detailed analysis about the security as well as the decryption failure rate (DFR) of the parameter sets can be found in Sections 2 and 3. Table 1 lists the main parameters, and Table 2 list the choices of the RM and the RS components of the concatenated codes for each parameter set.

Moreover, Table 3 show the instantiations of the symmetric cryptographic primitives used in QUBE. We introduce two primitive schemes for each parameter set, one using the SHA-3 hash family—for better comparison to the HQC algorithm—and another using the SM3 hash family provided by the NGCC program. For cells marked by stars in the table, we acknowledge that the primitive

QUBE.PKE.Enc($\text{pk}_{\text{PKE}}, m, \rho_c$) $\rightarrow c_{\text{PKE}}$:

Input: public key $\text{pk}_{\text{PKE}} \in \mathbb{B}^{\lambda/8+2 \cdot \lceil r/8 \rceil}$, message $m \in \mathbb{B}^{\lambda/8}$, randomness $\rho_c \in \mathbb{B}^{\lambda/8}$.

Output: ciphertext $c_{\text{PKE}} \in \mathbb{B}^{\lceil r/8 \rceil + L/8}$.

1. parse $\rho_0 \parallel s = \text{pk}_{\text{PKE}}$ where $\rho_0 \in \mathbb{B}^{\lambda/8}$ and $s \in \mathbb{B}^{2 \cdot \lceil r/8 \rceil}$
2. $\mathbf{H} = (\mathbf{h}_1, \mathbf{h}_2) = \text{GenUniformVector}(\rho_0, (r, r)) \in \mathcal{R}^2$
3. $\mathbf{S} = (\mathbf{s}_1, \mathbf{s}_2) = \text{Unpack}(s, (r, r))$
4. $(\mathbf{f}_1, \mathbf{f}_2, \mathbf{e}, \mathbf{g}) = \text{GenShortVector}(\rho_c, r, (w_f, w_f, w_e, w_g)) \in \mathcal{S}_{(w_f, w_f, w_e, w_g)}^r$,
and write $\mathbf{F} = (\mathbf{f}_1, \mathbf{f}_2) \in \mathcal{R}^2$
5. $\mathbf{u} = \mathbf{H} \cdot \mathbf{F}^\top + \mathbf{g}$, i.e., $\mathbf{u} = \mathbf{h}_1 \cdot \mathbf{f}_1 + \mathbf{h}_2 \cdot \mathbf{f}_2 + \mathbf{g}$
6. $\mathbf{m} = \text{Unpack}(m, \lambda)$
7. $\mathbf{v} = \text{trunc}(\mathbf{S} \cdot \mathbf{F}^\top + \mathbf{e}, L) + \mathcal{C}.\text{Encode}(\mathbf{m})$, where $\mathbf{S} \cdot \mathbf{F}^\top$ means $\mathbf{s}_1 \mathbf{f}_1 + \mathbf{s}_2 \mathbf{f}_2$
8. $c_{\text{PKE}} = \text{Pack}(\mathbf{u}) \parallel \text{Pack}(\mathbf{v})$
9. return c_{PKE}

Fig. 2: The encryption of QUBE.PKE

QUBE.PKE.Dec($\text{sk}_{\text{PKE}}, c_{\text{PKE}}$) $\rightarrow m$:

Input: private key $\text{sk}_{\text{PKE}} \in \mathbb{B}^{\lambda/8}$, ciphertext $c_{\text{PKE}} \in \mathbb{B}^{\lceil r/8 \rceil + L/8}$.

Output: message $m \in \mathbb{B}^{\lambda/8}$.

1. identify $\rho_1 = \text{sk}_{\text{PKE}}$
2. $\mathbf{y} = \text{GenShortVector}(\rho_1, r, w_y) \in \mathcal{S}_{w_y}^r$
3. parse $u \parallel v = c_{\text{PKE}}$ where $u \in \mathbb{B}^{\lceil r/8 \rceil}$ and $v \in \mathbb{B}^{L/8}$
4. $\mathbf{u} = \text{Unpack}(u, r)$, $\mathbf{v} = \text{Unpack}(v, L)$
5. $\mathbf{m} = \mathcal{C}.\text{Decode}(\mathbf{v} - \text{trunc}(\mathbf{y} \cdot \mathbf{u}, L))$
6. return $\text{Pack}(\mathbf{m})$

Fig. 3: The decryption of QUBE.PKE

used does not meet the required security level, while we have no alternative, more secure choice but to use those best existing primitive. All primitives are domain-separated by appending a distinct tag to the end of the input. For primitives whose outputs are longer than the expected length, the output is simply truncated to fit the length.

Table 4 lists the polynomial multiplication methods we use and recommend for each parameter set, and each implementation version. We note that different polynomial multiplication methods do not affect the design or specification of the QUBE algorithm, but merely act as speed optimizations.

1.6 Sizes of Keys and Ciphertexts

Here we present the size efficiency of QUBE. To recall from Figures 4 to 6, the public key, private key, and ciphertext of QUBE have sizes (in bytes) $\lambda/8 + 2 \cdot$

QUBE.KEM.KeyGen $((\rho, \rho_k)) \rightarrow (\text{pk}, \text{sk})$:

Input: randomness $\rho \in \mathbb{B}^{\lambda/8}$, randomness $\rho_k \in \mathbb{B}^{\lambda/8}$.
Output: public key $\text{pk} \in \mathbb{B}^{\lambda/8+2 \cdot \lceil r/8 \rceil}$, private key $\text{sk} \in \mathbb{B}^{3\lambda/8+2 \cdot \lceil r/8 \rceil}$.

1. $(\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}) = \text{QUBE.PKE.KeyGen}(\rho)$
2. $\text{pk} = \text{pk}_{\text{PKE}}$
3. $\text{sk} = \text{sk}_{\text{PKE}} \parallel \text{pk} \parallel \rho_k$
4. return pk, sk

Fig. 4: The key generation of QUBE.KEM

QUBE.KEM.Encaps $(\text{pk}, m, \sigma) \rightarrow (k, c)$:

Input: public key $\text{pk} \in \mathbb{B}^{\lambda/8+2 \cdot \lceil r/8 \rceil}$, randomness $m \in \mathbb{B}^{\lambda}$, salt $\sigma \in \mathbb{B}^{\lambda}$.
Output: shared secret $k \in \mathbb{B}^{\lambda}$, ciphertext $c \in \mathbb{B}^{\lceil r/8 \rceil + (L+\lambda)/8}$.

1. $(k, \rho_c) = \text{G}(m \parallel \sigma \parallel \text{H}(\text{pk}))$
2. identify $\text{pk}_{\text{PKE}} = \text{pk}$
3. $c_{\text{PKE}} = \text{QUBE.PKE.Enc}(\text{pk}_{\text{PKE}}, m, \rho_c)$
4. $c = c_{\text{PKE}} \parallel \sigma$
5. return k, c

Fig. 5: The encapsulation of QUBE.KEM

$\lceil r/8 \rceil$, $3\lambda/8+2 \cdot \lceil r/8 \rceil$, and $\lceil r/8 \rceil + (L+\lambda)/8$, respectively, and QUBE establishes $\lambda/8$ bytes of shared secret. Table 5 lists the resulting sizes for different parameter sets of QUBE presented above.

2 Design Rationale

2.1 Code-based Hard Computational Problems

In this section, we provide the hard computational problems on which the security of QUBE relies. These are all standard code-based problems. We start with the *syndrome decoding* (SD) problem, which is often regarded as the most classical hard problem in code-based cryptography.

Definition 1 (Syndrome Decoding (SD) Problem). Let $\mathbf{H} \in \mathbb{F}_2^{(n-k) \times n}$ be a parity-check matrix of a binary $[n, k]$ -linear code $\mathcal{C}$, $\mathbf{s} \in \mathbb{F}_2^{n-k}$ be a syndrome, and $w \in \mathbb{Z}_{\geq 0}$ be a nonnegative integer. The syndrome decoding (SD) problem, denoted by $\text{SD}(n, k, w)$, asks to find an error $\mathbf{e} \in \mathbb{F}_2^n$ such that $\mathbf{H}\mathbf{e}^\top = \mathbf{s}^\top$ and its Hamming weight is $\text{wt}(\mathbf{e}) = w$.

The SD problem has been proven NP-hard in [3] and thus plays a fundamental role in code-based cryptography. However, direct application of SD problem usually leads to large public key sizes. Many variants of the SD problem have

QUBE.KEM.Decaps(sk, c) $\rightarrow$ k:

Input: private key $\mathbf{sk} \in \mathbb{B}^{3\lambda/8+2\cdot\lceil r/8 \rceil}$, ciphertext $c \in \mathbb{B}^{\lceil r/8 \rceil+(L+\lambda)/8}$.
Output: shared secret $k \in \mathbb{B}^\lambda$.

1. parse $\mathbf{sk}_{\text{PKE}} \parallel \mathbf{pk} \parallel \rho_k = \mathbf{sk}$ where $\mathbf{sk}_{\text{PKE}} \in \mathbb{B}^{\lambda/8}$, $\mathbf{pk} \in \mathbb{B}^{\lambda/8+2\cdot\lceil r/8 \rceil}$, and $\rho_k \in \mathbb{B}^{\lambda/8}$
2. parse $c_{\text{PKE}} \parallel \sigma = c$ where $c_{\text{PKE}} \in \mathbb{B}^{\lceil r/8 \rceil+L/8}$ and $\sigma \in \mathbb{B}^{\lambda/8}$
3. $m' = \text{QUBE.PKE.Dec}(\mathbf{sk}_{\text{PKE}}, c_{\text{PKE}})$
4. $(k', c') = \text{QUBE.KEM.Enc}(\mathbf{pk}, m')$
5. $k^* = \text{J}(\rho_k \parallel c \parallel \text{H}(\mathbf{pk}))$
6. if $c' = c$ then return k'
7. else return k^*

Fig. 6: The decapsulation of QUBE.KEM

Table 1: Main parameters of QUBE.

Instance	r	w_x	w_y	w_f	w_g	w_e
QUBE-128	13109	74	76	29	27	83
*QUBE-192	25357	110	113	43	39	121
QUBE-256	41651	146	151	57	52	162
QUBE-384	82757	219	226	85	78	244
QUBE-512	135851	291	301	113	103	324

been proposed in various cryptographic scenarios to tackle the issue about large sizes. For example, people turn to quasi-cyclic codes to compress key sizes, which leads to the variant called *quasi-cyclic syndrome decoding* (QCSD) problem. In QUBE, we utilize a generalized version of the QCSD problem. We first define a specific QCSD distribution as follows.

Definition 2 ((s, t)-QCSD Distribution). For positive integers $s \geq t$ and vector $\mathbf{n} = (n_1, \dots, n_s)$, $\mathbf{w} = (w_1, \dots, w_s) \in \mathbb{Z}_{\geq 0}^s$, the (s, t)-QCSD($\mathbf{n}, \mathbf{w}$) distribution samples uniformly at random a parity-check matrix $\mathbf{H} \in \mathbb{F}_2^{(s-t)r \times sr}$ of a systematic QC code of index (s, t) (following the form of Equation (1), to recall) together with vectors $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_s) \xleftarrow{\$} \mathcal{S}_{\mathbf{w}}^{\mathbf{n}}$, and outputs $(\mathbf{H}, \mathbf{H}\mathbf{x}^\top)$.

Definition 3 (Computational (s, t)-QCSD Problem). Given an instance $(\mathbf{H}, \mathbf{y}^\top) \in \mathbb{F}_2^{(s-t)r \times sr} \times \mathbb{F}_2^{(s-t)r}$ sampled from the (s, t)-QCSD distribution, the (s, t)-QCSD($\mathbf{n}, \mathbf{w}$) problem asks to find $\mathbf{x} \in \mathcal{S}_{\mathbf{w}}^{\mathbf{n}}$ such that $\mathbf{H}\mathbf{x}^\top = \mathbf{y}^\top$.

When $n_i = r$ are all equal, we simply write the problem as (s, t)-QCSD($r, \mathbf{w}$). Moreover, if $w_i = w$ are all equal, the errors are said to be balanced, and the corresponding problem is called the balanced QCSD problem, simply denoted by (s, t)-QCSD(r, w); otherwise, they are referred to as unbalanced errors and the corresponding problem as the unbalanced QCSD problem.

Table 2: Codes used in QUBE. “RS short.” shows how much the dimension gets shortened in shortened RS code $[n_1, k_1, d_1]$; “RM mult.” shows the duplication multiplicity in duplicated RM code $[n_2, k_2, d_2]$.

Instance	RS short.	$[n_1, k_1, d_1]$	RM mult.	$[n_2, k_2, d_2]$	$L = n_1 \cdot n_2$
QUBE-128	237 – 221	[34, 16, 19]	3	[384, 8, 192]	13056
*QUBE-192	213 – 189	[66, 24, 43]	3	[384, 8, 192]	25344
QUBE-256	179 – 147	[108, 32, 77]	3	[384, 8, 192]	41472
QUBE-384	175 – 127	[128, 48, 81]	5	[640, 8, 320]	81920
QUBE-512	169 – 105	[150, 64, 87]	7	[896, 8, 448]	134400

 Table 3: Symmetric cryptographic primitives used in QUBE. $[\star]$ marks “placeholder” hash functions that technically do not meet the corresponding requirements about security levels.

Instance	H and J	PRG and G	XOF
QUBE-128-SHA3	SHA3-224	SHA3-256	SHAKE-128
*QUBE-192-SHA3	SHA3-224	SHA3-384	SHAKE-256
QUBE-256-SHA3	SHA3-256	SHA3-512	SHAKE-256
QUBE-384-SHA3	SHA3-384	SHAKE-256 $[\star]$	SHAKE-256 $[\star]$
QUBE-512-SHA3	SHA3-512	SHAKE-256 $[\star]$	SHAKE-256 $[\star]$
QUBE-128-SM3	SM3-256	SM3-256	SM3-XOF
*QUBE-192-SM3	SM3-256	SM3-512 $[\star]$	SM3-XOF
QUBE-256-SM3	SM3-256	SM3-512 $[\star]$	SM3-XOF
QUBE-384-SM3	SM3-512 $[\star]$	SM3-768 $[\star]$	SM3-XOF $[\star]$
QUBE-512-SM3	SM3-512 $[\star]$	SM3-1024 $[\star]$	SM3-XOF $[\star]$

The above search problem has the following decisional counterpart. While to get rid of trivial distinguishers, we impose a parity condition on the syndrome, defined as follows. For bit $a \in \mathbb{F}_2$, we define the finite set $P_a^r = \{\mathbf{h} \in \mathbb{F}_2^r \mid \mathbf{h}(1) = \sum_i h_i = a \pmod{2}\}$, and for vector $\mathbf{a} \in \mathbb{F}_2^\ell$, we define the vectorized version $P_{\mathbf{a}}^{\ell r} = \{(\mathbf{h}_1, \dots, \mathbf{h}_\ell) \mid \mathbf{h}_i \in P_{a_i}^r\}$. Moreover, for matrix $\mathbf{A} = (a_{i,j}) \in \mathbb{F}_2^{(s-t) \times t}$, we define the finite set

$$P_{\mathbf{A}}^{(s-t)r \times sr} = \left\{ \begin{pmatrix} \mathbf{I}_r & \mathbf{0} & \cdots & \mathbf{0} & \text{rot}(\mathbf{h}_{1,1}) & \cdots & \text{rot}(\mathbf{h}_{1,t}) \\ \mathbf{0} & \mathbf{I}_r & \ddots & \vdots & \text{rot}(\mathbf{h}_{2,1}) & \cdots & \text{rot}(\mathbf{h}_{2,t}) \\ \vdots & \ddots & \ddots & \mathbf{0} & \vdots & \cdots & \vdots \\ \mathbf{0} & \cdots & \mathbf{0} & \mathbf{I}_r & \text{rot}(\mathbf{h}_{s-t,1}) & \cdots & \text{rot}(\mathbf{h}_{s-t,t}) \end{pmatrix} \in \mathbb{F}_2^{sr \times tr} \mid \right. \\ \left. \mathbf{h}_{i,j} \in P_{a_{i,j}}^r, 1 \leq i \leq s-t, 1 \leq j \leq t \right\}.$$

Table 4: Polynomial multiplication methods recommended in QUBE, for reference version implementation and time-optimized version implementation.

Instance	QUBE-128	QUBE-256	QUBE-384	QUBE-512
Method	K2	TC3-K2	TC3-K2	TC3-K2

Table 5: Sizes (in bytes) of keys and ciphertexts in QUBE, compared to those of HQC.

Instance	$ \mathbf{sk} $	$ \mathbf{pk} $	$ c $	$ \mathbf{pk} + c $	$ k $
QUBE-128	3,326	3,294	3,287	6,581	16
*QUBE-192	6,412	6,364	6,362	12,726	24
QUBE-256	10,510	10,446	10,423	20,869	32
QUBE-384	20,834	20,738	20,633	41,371	48
QUBE-512	34,156	34,028	33,846	67,874	64
HQC-1	2,321	2,241	4,433	6,674	32
HQC-3	4,602	4,514	8,978	13,492	32
HQC-5	7,333	7,237	14,421	21,658	32

We then plant the above parity conditions into the definition of QCSD distribution as follows.

Definition 4 (((s, t) -QCSD Distribution with Parity). For positive integers $s \geq t$ and r , vector $\mathbf{w} = (w_1, \dots, w_s) \in \mathbb{Z}_{\geq 0}^s$, and matrix $\mathbf{A} \in \mathbb{F}_2^{(s-t) \times t}$, the (s, t) -QCSD($r, \mathbf{w}; \mathbf{A}$) distribution samples uniformly at random a parity-check matrix $\mathbf{H} \xleftarrow{\$} P_{\mathbf{A}}^{(s-t)r \times sr}$ together with a vector $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_s) \xleftarrow{\$} \mathcal{S}_{\mathbf{w}}^r$, and outputs $(\mathbf{H}, \mathbf{H}\mathbf{x}^\top)$.

We then define the decisional version of the QCSD problem.

Definition 5 (Decisional (s, t) -QCSD Problem (with Parity)). Let $\mathbf{w} \in \mathbb{Z}_{\geq 0}^s$, $\mathbf{A} \in \mathbb{F}_2^{(s-t) \times t}$ be as defined above, and let $\mathbf{b} = (b_1, \dots, b_{s-t})$ where $b_i := w_i + \sum_{j=t+1}^s w_j a_{i,j}$ for $1 \leq i \leq s-t$. The decisional (s, t) -QCSD($r, \mathbf{w}; \mathbf{A}$) problem asks to decide with non-negligible advantage whether the input instance $(\mathbf{H}, \mathbf{y}^\top)$ comes from the (s, t) -QCSD($r, \mathbf{w}; \mathbf{A}$) distribution, or the uniform distribution over $P_{\mathbf{A}}^{(s-t)r \times sr} \times P_{\mathbf{b}}^{(s-t)r}$.

When $s = t = 1$, the problem degenerates to the plain, standard SD problem. When $w_i = w$ are all equal and $t = 1$, it becomes the s -QCSD problem involved in HQC [1].

Next, we define the new concept of *combinatorial gap*, which is an important contribution of the analysis of QUBE.

Definition 6 (Combinatorial Gap). For a positive integer s and two vectors $\mathbf{n} = (n_1, \dots, n_s), \mathbf{w} = (w_1, \dots, w_s) \in \mathbb{Z}_{\geq 0}^s$, the combinatorial gap for $\mathbf{n}$ and $\mathbf{w}$ is defined by

$$\begin{aligned} \text{CombGap}(\mathbf{n}, \mathbf{w}) &= \log_2 \left(\frac{\left(\sum_{i=1}^s n_i \right)}{\left(\sum_{i=1}^s w_i \right)} \prod_{i=1}^s \binom{n_i}{w_i} \right) \\ &= \log_2 \left(\frac{\sum_{i=1}^s n_i}{\sum_{i=1}^s w_i} \right) - \sum_{i=1}^s \log_2 \binom{n_i}{w_i}. \end{aligned}$$

It turns out that combinatorial gap is always nonnegative, and intuitively speaking, larger s gives larger combinatorial gap. In QUBE we only consider the case where $n_i = r$ are all equal, and simplify the notation as $\text{CombGap}(r, \mathbf{w})$.

Finally, we present a result about reductions between unbalanced QCSD problems and balanced QCSD problems to showcase their relationship; the reductions are however not the tightest, and not as strong as combining the above new concept of combinatorial gap with the attacks we later consider in Section 3.2. The general idea of the reduction is the following. Consider the QCSD problem (s, t) -QCSD($r, \mathbf{w}$). Let $W := \max_i w_i$ be the maximum weight among the entries. Given an instance $(\mathbf{H}, \mathbf{y}^\top)$ of the problem, we can randomly generate $\mathbf{z} = (\mathbf{z}_1, \dots, \mathbf{z}_s)$ such that $\text{wt}(\mathbf{z}_i) = W - w_i$. Then, we have that $(\mathbf{H}, \mathbf{y}^\top + \mathbf{H}\mathbf{z}^\top)$ is an instance of the balanced QCSD problem (s, t) -QCSD(r, W) (with proper probability). Similarly, we can also “pad” an instance of a balanced problem into an instance of an unbalanced problem.

Proposition 1. For positive integers $s \geq t$ and r and vector $\mathbf{w} \in \mathbb{Z}_{\geq 0}^s$, the following chain of reductions holds:

$$(s, t)\text{-QCSD}(r, \min_i w_i) \leq (s, t)\text{-QCSD}(r, \mathbf{w}) \leq (s, t)\text{-QCSD}(r, \max_i w_i).$$

To date, there is no general complexity lower bound result for the (s, t) -QCSD problem, while there is neither any successful attack. We make the assumption that the (s, t) -QCSD problem enjoys similar hardness as the plain SD problem.

In addition, similar to the case of HQC, we do not consider the SD variants about decoding with erasures.

2.2 QCSD Problems underlying QUBE

There are two QCSD problems embedded in QUBE. (Recall Figures 1 and 2.)

In key generation, we have equation $\mathbf{S} = \mathbf{X} + \mathbf{y} \cdot \mathbf{H}$, which can be rewritten as:

$$\begin{pmatrix} \mathbf{I}_r & \mathbf{0} & \text{rot}(\mathbf{h}_1) \\ \mathbf{0} & \mathbf{I}_r & \text{rot}(\mathbf{h}_2) \end{pmatrix} \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{y} \end{pmatrix} = \begin{pmatrix} \mathbf{s}_1 \\ \mathbf{s}_2 \end{pmatrix}. \quad (2)$$

Here $(\mathbf{x}_1, \mathbf{x}_2, \mathbf{y}) \in \mathcal{R}^3$ and have Hamming weights $\mathbf{w}_{\text{pk}} = (w_x, w_x, w_y)$. This matches exactly with unbalanced $(3, 1)$ -QCSD($r, \mathbf{w}_{\text{pk}}$) problem, which we also refer to as the PK problem.

Table 6: The concrete bits of security levels of the QCSD problems in QUBE, including both security under classical attacks and under quantum attacks.

Instance	QUBE-128 *QUBE-192 QUBE-256 QUBE-384 QUBE-512
PK Problem, Class.	144.77208.68273.19402.58530.55
PK Problem, Quant.	102.44136.73170.71237.87303.73
CT Problem, Class.	180.78259.00341.17505.38666.27
CT Problem, Quant.	120.88162.29205.09289.60371.91
PK Comb. Gap	8.108.699.149.7510.22
CT Comb. Gap	44.2059.7977.45112.44145.84
Overall Class.	136.5199.2263.7392.8520.3
Overall Quant.	98.3132.3166.1232.9298.6

Next, in encryption/encapsulation, we have equations $\mathbf{u} = \mathbf{H}\mathbf{F}^\top + \mathbf{g}$ and $\mathbf{v} = \mathbf{S}\mathbf{F}^\top + \mathbf{e} + \mathcal{C}.\text{Encode}(\mathbf{m})$ (omitting the truncation operation), which similarly can be rewritten into:

$$\begin{pmatrix} \mathbf{I}_r & \mathbf{0} & \text{rot}(\mathbf{h}_1) & \text{rot}(\mathbf{h}_2) \\ \mathbf{0} & \mathbf{I}_r & \text{rot}(\mathbf{s}_1) & \text{rot}(\mathbf{s}_2) \end{pmatrix} \begin{pmatrix} \mathbf{g} \\ \mathbf{e} \\ \mathbf{f}_1 \\ \mathbf{f}_2 \end{pmatrix} = \begin{pmatrix} \mathbf{u} \\ \mathbf{v} - \mathcal{C}.\text{Encode}(\mathbf{m}) \end{pmatrix}. \quad (3)$$

Here $(\mathbf{g}, \mathbf{e}, \mathbf{f}_1, \mathbf{f}_2) \in \mathcal{R}^4$ and have Hamming weights $\mathbf{w}_{\text{ct}} = (w_g, w_e, w_f, w_f)$. This matches exactly with unbalanced $(4, 2)$ -QCSD($r, \mathbf{w}_{\text{ct}}$) problem, which we also refer to as the CT problem.

In Section 3.1 we establish more formally the fact that the security of QUBE can be derived from the hardness of the PK and CT problems. In Section 3.2 we consider state-of-the-art attacks on the two problems, combined with our new concept about combinatorial gap. The results about concrete bits of security levels are collected in Table 6. It can be observed that for each security level and the corresponding parameter set, both classical and quantum security levels satisfy the required lower bound.

2.3 Analysis of Decryption Failure Rate

In this section, we analyze the decryption failure rate (DFR) of QUBE. We re-emphasize that the Fujisaki-Okamoto transform wraps an IND-CPA secure PKE scheme into an IND-CCA secure KEM scheme under the assumption that the PKE scheme has a negligible DFR lower than $2^{-\lambda}$ [9], where λ is the security parameter.

To recall, in decryption/decapsulation of QUBE, we compute

$$\begin{aligned} \mathbf{v} - \mathbf{y} \cdot \mathbf{u} &= (\mathbf{X} + \mathbf{y} \cdot \mathbf{H}) \cdot \mathbf{F}^\top + \mathbf{e} + \mathcal{C}.\text{Encode}(\mathbf{m}) - \mathbf{y} \cdot (\mathbf{H} \cdot \mathbf{F}^\top + \mathbf{g}) \\ &= \mathbf{X} \cdot \mathbf{F}^\top - \mathbf{y} \cdot \mathbf{g} + \mathbf{e} + \mathcal{C}.\text{Encode}(\mathbf{m}) \\ &= \mathbf{x}_1 \cdot \mathbf{f}_1 + \mathbf{x}_2 \cdot \mathbf{f}_2 - \mathbf{y} \cdot \mathbf{g} + \mathbf{e} + \mathcal{C}.\text{Encode}(\mathbf{m}). \end{aligned}$$

(Here we again omit the truncation.) We then call $\mathcal{C}.$ Decode to handle the total error term $\hat{\mathbf{e}} = \mathbf{x}_1 \cdot \mathbf{f}_1 + \mathbf{x}_2 \cdot \mathbf{f}_2 - \mathbf{y} \cdot \mathbf{g} + \mathbf{e}$.

In QUBE, a concatenated code $\mathcal{C}$ is used to encode and decode the message/shared secret, which consists of an $[n_2, 8, n_2/2]$ Reed-Muller code as the internal code and an $[n_1, k_1, n_1 - k_1 + 1]$ Reed-Solomon code as the external code. Since we use the same family of codes as in HQC, we directly borrow its calculations of DFR and include them as follows.

Firstly, the following proposition describes the probability distribution of multiplying two vectors as polynomials in $\mathcal{R}$.

Proposition 2 ([10, Proposition 6.1.1]). *Suppose random vectors $\mathbf{u} \xleftarrow{\$} \mathcal{S}_{w_u}^r$ and $\mathbf{v} \xleftarrow{\$} \mathcal{S}_{w_v}^r$ are independent. For $\mathbf{z} = \mathbf{u} \cdot \mathbf{v} \in \mathcal{R}$, each of its coordinate z_i follows a Bernoulli distribution with parameter:*

$$\tilde{p}(r, w_u, w_v) := \frac{1}{\binom{r}{w_u} \binom{r}{w_v}} \cdot \sum_{\substack{1 \leq t \leq \min\{w_u, w_v\} \\ t \text{ odd}}} \binom{r}{t} \binom{r-t}{w_u-t} \binom{r-w_u}{w_v-t}.$$

We generalize [10, Proposition 6.1.2] to get the following proposition that describes the probability of summing up Bernoulli variables.

Proposition 3. *Suppose random vectors $\mathbf{z}_1, \dots, \mathbf{z}_\ell$ are independent, and each coordinate of $\mathbf{z}_i$ follows a Bernoulli distribution with parameter p_i . Then each entry of their sum $\mathbf{s} = \sum_i \mathbf{z}_i$ follows a Bernoulli distribution with parameter:*

$$p(p_1, \dots, p_\ell) := \sum_{\substack{\mathbf{b} \in \{0,1\}^\ell \\ \text{wt}(\mathbf{b}) \text{ odd}}} \prod_{i=1}^{\ell} p_i^{b_i} (1-p_i)^{1-b_i}.$$

Combining Propositions 2 and 3 (while approximating $\mathbf{e}$ by viewing it as a Bernoulli vector with parameter w_e/r), we can calculate the Bernoulli parameter $\hat{p}$ of the total error $\hat{\mathbf{e}}$. We then feed $\hat{p}$ along the following two existing results to calculate (upper bound on) the final DFR.

Proposition 4 ([10, Proposition 6.1.4]). *Let p be the transition probability of the binary symmetric channel. Then the DFR of an $[n_{\text{in}}, 8, d_{\text{in}}]$ -Reed-Muller code is bounded from above by:*

$$\begin{aligned} p_{\text{in}} &= \sum_{i=d_{\text{in}}/2}^{n_{\text{in}}} C_i p^i (1-p)^{n_{\text{in}}-i}, \text{ where} \\ C_i &= \min \left\{ \binom{n_{\text{in}}}{i}, \frac{255}{2} \binom{d_{\text{in}}}{d_{\text{in}}/2} \binom{d_{\text{in}}}{i-d_{\text{in}}/2} + 255 \sum_{j=d_{\text{in}}/2+1}^{d_{\text{in}}} \binom{d_{\text{in}}}{j} \binom{d_{\text{in}}}{i-j} \right. \\ &\quad \left. + \frac{1}{2} \binom{255}{2} \sum_{j=0}^{d_{\text{in}}/2} \binom{d_{\text{in}}/2}{j}^3 \binom{d_{\text{in}}/2}{i-d_{\text{in}}+j} \right\}. \end{aligned}$$

Table 7: Decryption failure rates of QUBE.

Instance	QUBE-128	*QUBE-192	QUBE-256	QUBE-384	QUBE-512
DFR Level	137.5	200.1	264.3	392.4	520.6

Proposition 5 ([10, Theorem 6.1]). *For the concatenated code using an $[n_e, k_e, d_e]$ -Reed-Solomon code (where $d_e = 2\delta_e + 1$ is odd) as the external code and a Reed-Muller code as the internal code, with (upper bound on) internal DFR p_{in} , the external DFR is bounded from above by:*

$$\sum_{j=\delta_e+1}^{n_e} \binom{n_e}{j} p_{\text{in}}^j (1 - p_{\text{in}})^{n_e-j}.$$

Applying the above chain of tools to the parameter set of QUBE, we get the DFR of QUBE, listed in Table 7 and converted to bit level (i.e., $\log_2(1/[\text{DFR}])$). Note that every parameter set has its DFR level safely above the security parameter, as expected.

Dynamics of parameter selection. We conclude the section by briefing a couple of principles in selecting the parameters of QUBE. Overall, we have two categories of constraints, that the PK problem ((3, 1)-QCSD) and the CT problem ((4, 2)-QCSD) are hard enough, and that the DFR level is high enough. The former constraint desires for larger weights for the SD problems to be harder in general, and more balanced weights so that the combinatorial gap is smaller. While the latter constraint desires for smaller weights so that the total error is shorter, and more unbalanced weights also for the same purpose (as the total error involves cross-product $\mathbf{X} \cdot \mathbf{F}^\top + \mathbf{y} \cdot \mathbf{g}$, the more unbalanced $(\mathbf{X}, \mathbf{y})$ or $(\mathbf{F}, \mathbf{g})$ is, the “smaller” the resulting product would be). It is immediate that the two constraints are asking for contradicting moves, and the parameter selection process is to look for the best balancing between them. In addition, we consider extra constraints on the parameters such as those in Section 3.3.

3 Security Analysis

3.1 Reduction-based Security

In this section, we first present the proof of the IND-CPA security of QUBE.PKE assuming the hardness of the relevant problems introduced in Section 2.2. We then demonstrate that the IND-CCA security of QUBE.KEM is guaranteed by the Fujisaki-Okamoto transform.

IND-CPA security of QUBE.PKE under ROM. The notion of IND-CPA security [12], captured by the games $\mathbf{Exp}_{\mathcal{E}, \mathcal{A}}^{\text{ind-b}}(\lambda)$ below, states that no adversary can

efficiently guess which of two plaintexts (of its choice) has been encrypted. And we define the advantage of an adversary $\mathcal{A}$ winning in the game $\mathbf{Exp}_{\mathcal{E},\mathcal{A}}^{\text{ind-}b}(\lambda)$ as:

$$\text{Adv}_{\mathcal{E},\mathcal{A}}^{\text{ind}}(\lambda) = \left| \Pr[\mathbf{Exp}_{\mathcal{E},\mathcal{A}}^{\text{ind-}1}(\lambda) = 1] - \Pr[\mathbf{Exp}_{\mathcal{E},\mathcal{A}}^{\text{ind-}0}(\lambda) = 1] \right|.$$

We say that an encryption scheme is IND-CPA secure if for every PPT adversary $\mathcal{A}$, the advantage $\text{Adv}_{\mathcal{E},\mathcal{A}}^{\text{ind}}(\lambda)$ is negligible in λ .

$\mathbf{Exp}_{\mathcal{E},\mathcal{A}}^{\text{ind-}b}(\lambda)$:

1. $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{param})$.
2. $(m_0, m_1) \leftarrow \mathcal{A}(\text{Find} : \text{pk})$.
3. $c^* \leftarrow \text{Enc}(\text{pk}, m_b)$.
4. $b' \leftarrow \mathcal{A}(\text{GUESS} : c^*)$.
5. Return b' .

Theorem 1. *The scheme QUBE.PKE presented in Figures 1 to 3 is IND-CPA secure under the assumption that both decisional $(3, 1)$ -QCSD($r, (w_x, w_x, w_y)$) and decisional $(4, 2)$ -QCSD($r, (w_g, w_e, w_f, w_f)$) are hard to solve.*

Proof. The proof is similar to that in [1], with the difference being that the difficult problems are the $(3, 1)$ -QCSD and $(4, 2)$ -QCSD instances; see Equations (2) and (3) for the instances.

We prove the security of our scheme via a hybrid argument. Specifically, we construct a sequence of games that transitions from a game that encrypts message m_0 to a game that encrypts message m_1 . Any adversary that distinguishes between these two games can be used to build a simulator that breaks the either the $(3, 1)$ -QCSD or the $(4, 2)$ -QCSD assumption with comparable running time.

Game $\mathcal{G}_{\mathcal{E},\mathcal{A}}^1(\lambda)$: This is the game where message m_0 is correctly encrypted, defined as follows. Here for convenience, we ignore the difference between storing a seed and storing the actual data generated from the seed when defining the game, as well as the difference between packed or unpacked formats.

$\mathcal{G}_{\mathcal{E},\mathcal{A}}^1(\lambda)$:

1. $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$ with $\text{pk} = (\mathbf{H}, \mathbf{S} = \mathbf{X} + \mathbf{y} \cdot \mathbf{H})$ and $\text{sk} = \mathbf{y}$.
2. $(m_0, m_1) \leftarrow \mathcal{A}(\text{Find} : \text{pk})$.
3. $c^* \leftarrow \text{Enc}(\text{pk}, m_0) = (\mathbf{u}, \mathbf{v})$.
4. $b' \leftarrow \mathcal{A}(\text{GUESS} : c^*)$.
5. Return b' .

Game $\mathcal{G}_{\mathcal{E},\mathcal{A}}^2(\lambda)$: This game differs from $\mathcal{G}_{\mathcal{E},\mathcal{A}}^1(\lambda)$ in that the public key $\mathbf{pk}$ is no longer correlated to the secret key $\mathbf{sk} = \mathbf{y}$, by re-sampling a $\mathbf{S}'$ uniformly at random from the set of all strings sharing the same parity bits as the real $\mathbf{S}$.

$\mathcal{G}_{\mathcal{E},\mathcal{A}}^2(\lambda)$:

1. $(\mathbf{pk}_0, \mathbf{sk}) \leftarrow \text{KeyGen}()$ with $\mathbf{pk}_0 = (\mathbf{H}, \mathbf{S})$ and $\mathbf{sk} = \mathbf{y}$.
2. $\mathbf{S}' \xleftarrow{\$} P_{\mathbf{b}}^{2r}$, where $\mathbf{b}$ is calculated from the parity of $\mathbf{H}$ and the weights w_x, w_y , as defined in Definition 5 and following the structure of Equation (2).
3. $\mathbf{pk} \leftarrow (\mathbf{H}, \mathbf{S}')$.
4. $(m_0, m_1) \leftarrow \mathcal{A}(\text{Find} : \mathbf{pk})$.
5. $c^* \leftarrow \text{Enc}(\mathbf{pk}, m_0) = (\mathbf{u}, \mathbf{v})$.
6. $b' \leftarrow \mathcal{A}(\text{GUESS} : c^*)$.
7. Return b' .

Observe that $\mathcal{G}_{\mathcal{E},\mathcal{A}}^1(\lambda)$ and $\mathcal{G}_{\mathcal{E},\mathcal{A}}^2(\lambda)$ differ merely by the structure of $\mathbf{pk}$ —whether it has the (3,1)-QCSD structure with secret $\mathbf{y}$, or it has $\mathbf{S}'$ freshly sampled. This matches exactly with the (3,1)-QCSD problem (with parity). Hence we know $\mathcal{G}_{\mathcal{E},\mathcal{A}}^1(\lambda)$ and $\mathcal{G}_{\mathcal{E},\mathcal{A}}^2(\lambda)$ are indistinguishable assuming the hardness of that problem.

Note that now the private key $\mathbf{sk} = \mathbf{y}$ is never used in the game, and can be discarded from the game.

Game $\mathcal{G}_{\mathcal{E},\mathcal{A}}^3(\lambda)$: In this next game, we turn to re-randomize the ciphertexts and drop the QCSD structure given by $\mathbf{f}_i, \mathbf{g}, \mathbf{e}$.

$\mathcal{G}_{\mathcal{E},\mathcal{A}}^3(\lambda)$:

1. $(\mathbf{pk}_0, \mathbf{sk}) \leftarrow \text{KeyGen}()$ with $\mathbf{pk}_0 = (\mathbf{H}, \mathbf{S})$ and $\mathbf{sk} = \mathbf{y}$.
2. $\mathbf{S}' \xleftarrow{\$} P_{\mathbf{b}}^{2r}$, where $\mathbf{b}$ is described as above.
3. $\mathbf{pk} \leftarrow (\mathbf{H}, \mathbf{S}')$.
4. $(m_0, m_1) \leftarrow \mathcal{A}(\text{Find} : \mathbf{pk}_0)$.
5. $\mathbf{u} \xleftarrow{\$} P_{b_u}^r, \mathbf{v}' \xleftarrow{\$} P_{b_v}^r$, where b_u, b_v are calculated from the parity of $\mathbf{H}$ and the weights w_f, w_g, w_e , as defined in Definition 5 and following the structure of Equation (3).
6. $\mathbf{v} \leftarrow \text{trunc}(\mathbf{v}') + \mathcal{C}.\text{Encode}(m_0)$
7. $c^* \leftarrow (\mathbf{u}, \mathbf{v})$.
8. $\mathbf{b}' \leftarrow \mathcal{A}(\text{GUESS} : c^*)$.
9. Return b' .

Similarly, it can be observed that the difference between $\mathcal{G}_{\mathcal{E},\mathcal{A}}^2(\lambda)$ and $\mathcal{G}_{\mathcal{E},\mathcal{A}}^3(\lambda)$ is the structure of the ciphertext c —whether it has the (4,2)-QCSD structure with secrets $\mathbf{f}_i, \mathbf{g}, \mathbf{e}$ or it just has $\mathbf{u}, \mathbf{v}'$ freshly sampled. This matches exactly with

the (4, 2)-QCS problem (with parity). Hence we know $\mathcal{G}_{\mathcal{E}, \mathcal{A}}^2(\lambda)$ and $\mathcal{G}_{\mathcal{E}, \mathcal{A}}^3(\lambda)$ are indistinguishable assuming the hardness of that problem.

Due to the randomness of $\mathbf{v}'$, the ciphertext c^* now no longer depends on the plaintext m_0 , and hence if we start from a game that encrypts m_1 , and follow similar hybrids, we would reach the same game $\mathcal{G}_{\mathcal{E}, \mathcal{A}}^3(\lambda)$. This implies that the IND-CPA games $\mathbf{Exp}_{\mathcal{E}, \mathcal{A}}^{\text{ind-0}}(\lambda)$ and $\mathbf{Exp}_{\mathcal{E}, \mathcal{A}}^{\text{ind-1}}(\lambda)$ are indistinguishable, as desired.

IND-CCA security of QUBE.KEM under ROM. The above establishes the IND-CPA security of QUBE.PKE, and from Section 2.3, we know QUBE.PKE also has a sufficiently low DFR. As shown in [13], by applying the standard Fujisaki-Okamoto transform, the IND-CPA security of our QUBE.PKE tightly implies the IND-CCA2 security of the resulting KEM, which is QUBE.KEM as shown in Figures 4 to 6.

To recall, IND-CCA2 security differs from IND-CPA security in allowing the attacker access to a decryption oracle during both the Find and GUESS phases (with the restriction that the challenge ciphertext c^* cannot be queried to the decryption oracle). We omit the formal IND-CCA2 definition and security games here, as we rely in a black-box way on the (efficient) FO transform from [13].

Our construction employs the FO_m^χ transform proposed in [13]. Under the random oracle model (ROM), this transform enjoys the following tight reduction-based result.

Theorem 2 (FO transform under ROM [13, Theorems 3.2 and 3.6], simplified). *Let PKE be a public-key encryption scheme and let F be a hash function modeled as a random oracle. For any PPT adversary $\mathcal{A}$ that breaks the IND-CCA2 security of the KEM scheme $\text{KEM} := \text{FO}_m^\chi[\text{PKE}, F]$, there exists a PPT adversary $\mathcal{B}$ that breaks the IND-CPA security of PKE such that*

$$\text{Adv}_{\text{KEM}, \mathcal{A}}^{\text{IND-CCA}}(\lambda) \leq 3 \cdot \text{Adv}_{\text{PKE}, \mathcal{B}}^{\text{IND-CPA}}(\lambda) + (2q_F + q_D) \cdot \delta + \frac{3q_F}{|M|},$$

where δ is the decryption failure probability of PKE, q_F, q_D are the numbers of queries to the random oracle F and the decryption oracle D , respectively, and $|M|$ is the size of the message space of PKE.

IND-CCA security of QUBE.KEM under QROM. In addition, under the quantum random oracle model (QROM), [14] shows that the FO transform we employ enjoys the following tight reduction-based result.

Theorem 3 (FO transform under the QROM [14, Theorem 1], simplified). *Let PKE be a public-key encryption scheme, and let F be a hash function. For any QPT adversary $\mathcal{A}$ that breaks the IND-CCA2 security of the KEM scheme $\text{KEM} := \text{FO}_m^\chi[\text{PKE}, F]$, there exists a QPT adversary $\mathcal{B}$ that breaks the IND-CPA security of PKE such that*

$$\text{Adv}_{\text{KEM}, \mathcal{A}}^{\text{IND-CCA}}(\lambda) \leq 2\sqrt{(q_F + 1) \cdot \text{Adv}_{\text{PKE}, \mathcal{B}}^{\text{IND-CPA}}(\lambda) + \frac{2(q_F + 1)^2}{|M|}} + 4q_F\sqrt{\delta} + \frac{q_D}{2\lambda},$$

where δ is the decryption failure probability of PKE, q_F, q_D are the numbers of queries to the random oracle F and the decryption oracle D , respectively, and $|M|$ is the size of the message space of PKE.

3.2 Practical Security

In this section, we present the state-of-the-art attacks on the (QC)SD problems.

3.2.1 ISD Algorithms for QCSD Problems Here we explain the information set decoding (ISD) algorithms for the (s, t) -QCSD problem with unbalanced errors. ISD is currently the best algorithm for solving the SD problem. Based on ISD algorithms, we describe a blockwise ISD algorithm that exploits the characteristics of unbalanced errors within the blocks.

We first introduce the development of ISD algorithms. The ISD algorithm was originally proposed by Prange in 1962 for decoding binary cyclic codes [18]. Subsequently, a large number of sophisticated techniques and strategies have been developed to improve Prange's algorithm. Based on the different techniques employed, the current mainstream ISD algorithms can be classified into four categories:

- (1) Permutation-based ISD, with representative algorithms including the Prange and Lee-Brickell algorithms [15];
- (2) Enumeration-based ISD, with representative algorithms including Stern [21] and Dumer [5];
- (3) Representation-based ISD, with representative algorithms including MMT [16] and BJMM [2];
- (4) Nearest-neighbor-based ISD search, with representative algorithms including [17], [4], and [6].

The goal of the binary SD problem is to find a binary vector $\mathbf{r} \in \mathbb{F}_2^n$ with a matching syndrome and Hamming weight of w . We first describe the common framework for these major ISD algorithms, and then adapt these algorithms to the quasi-cyclic setting. The input to an ISD algorithm consists of integers n, k, w , a parity-check matrix $\mathbf{H} \in \mathbb{F}_2^{(n-k) \times n}$, and a syndrome $\mathbf{s} \in \mathbb{F}_2^{n-k}$. The ISD algorithm outputs a vector $\mathbf{e} \in \mathbb{F}_2^n$ satisfying $\mathbf{H}\mathbf{e}^\top = \mathbf{s}^\top$ and $\text{wt}(\mathbf{e}) = w$. The algorithm consists of four steps:

Step 1: Permutation. Randomly choose a permutation matrix $\mathbf{P} \in \mathbb{F}_2^{n \times n}$. The permuted instance is then $\mathbf{H}' = \mathbf{H}\mathbf{P}$, and the corresponding error vector is $\mathbf{e}' = \mathbf{P}^{-1}\mathbf{e}$, which redistributes the weight of the error, and such that $\mathbf{H}\mathbf{P}_{[n] \setminus [k+\ell]}$ is partially reversible.

Step 2: Partial Gaussian Elimination. Randomly choose an invertible matrix $\mathbf{Q} \in \mathbb{F}_2^{(n-k) \times (n-k)}$ such that

$$\mathbf{QHP} = \begin{pmatrix} \mathbf{H}_1 & \mathbf{0} \\ \mathbf{H}_2 & \mathbf{I}_{n-k-\ell} \end{pmatrix}, \mathbf{Qs}^\top = \begin{pmatrix} \hat{\mathbf{s}} \\ \tilde{\mathbf{s}} \end{pmatrix},$$

where $\mathbf{H}_1 \in \mathbb{F}_2^{\ell \times (k+\ell)}$, $\mathbf{H}_2 \in \mathbb{F}_2^{(n-k-\ell) \times (k+\ell)}$, $\mathbf{s}_1 \in \mathbb{F}_2^\ell$, $\mathbf{s}_2 \in \mathbb{F}_2^{n-k-\ell}$. Meanwhile, the error vector $\mathbf{e}$ is split into two vectors $\mathbf{e}_1 \in \mathbb{F}_2^{k+\ell}$ and $\mathbf{e}_2 \in \mathbb{F}_2^{n-k-\ell}$, i.e., $\mathbf{eP} = (\mathbf{e}_1^\top \mid \mathbf{e}_2^\top)$. Consequently, we have the following two equations:

$$\mathbf{H}_1 \mathbf{e}_1^\top = \mathbf{s}_1, \quad (4)$$

$$\mathbf{H}_2 \mathbf{e}_1^\top + \mathbf{e}_2^\top = \mathbf{s}_2. \quad (5)$$

Step 3: Sub-SD Solver. Solve the $(k+\ell, \ell, p)$ -SD subproblem, i.e., find an error vector $\mathbf{e}_1$ satisfying Equation (4) with $\text{wt}(\mathbf{e}_1) = p$.

Step 4: Check. For each solution $\mathbf{e}_2$, compute $\mathbf{e}_2^\top = \mathbf{s}_2^\top - \mathbf{H}_2 \mathbf{e}_1^\top$ and check whether $\text{wt}(\mathbf{e}_2) = w - p$. If so, output $\mathbf{e} = (\mathbf{e}_1 \mid \mathbf{e}_2)$; otherwise, return to Step 1.

We now briefly describe the time complexity of ISD, also referred to as the work factor (WF). Let T_{PG} denote the time complexity of the partial Gaussian elimination step, and let T_{sub} denote the time complexity of solving the sub-SD problem. Additionally, let P_{Succ} be the probability of successfully finding a solution with the single call of the search function, and

$$P_{\text{Succ}} = \frac{\binom{n-k-\ell}{w-p} \binom{k+\ell}{p}}{\binom{n}{w}}.$$

Then the (expected) time complexity of ISD can be expressed as

$$T_{\text{ISD}} = \frac{T_{\text{sub}}}{P_{\text{Succ}}}. \quad (6)$$

In brief, the difference among various ISD algorithms lies in the methods used to solve the sub-SD problem, i.e., the difference in T_{sub} . The main techniques employed include meet-in-the-middle attacks, representation techniques, subset sum computations, and nearest neighbor searches, leading to different computational complexities.

Additionally, the framework of [4], which employs nearest-neighbor search techniques, is slightly different from the above framework. However, the formula for evaluating the computational complexity and the formula for the probability of successfully finding a solution are similar to Equation (6) in the general framework described above. Therefore, we take that framework into consideration while omitting the detailed differences here.

In what follows, we show ISD algorithms on the (s, t) -QCSD($r, \mathbf{w}$) problem with unbalanced errors by adopting the common framework for the SD problem. The goal of this problem is to find a binary vector $\mathbf{e} = (\mathbf{e}_1, \dots, \mathbf{e}_s) \in \mathbb{F}_2^{rs}$ with the Hamming weight $\mathbf{w} = (w_1, \dots, w_s)$ and $w = w_1 + \dots + w_s$, meaning that for each $1 \leq i \leq s$, the w_i ones in $\mathbf{e}_i$ are uniformly distributed over r positions, such that $\mathbf{He} = \mathbf{s}$.

Similarly, we first apply a permutation matrix $\mathbf{P}$ to the columns of the parity-check matrix $\mathbf{H}$, so that the weight of the blockwise error $\mathbf{e}$ is redistributed. In this way, the structured nature of the blockwise error is weakened, and the aforementioned ISD algorithm can be used to solve the QCSD problem. However,

for the blockwise error case, the difference is that if P_{Succ} denotes the probability of successfully finding a solution in a single call to the search function, then

$$P_{\text{Succ}} = \frac{\binom{t \cdot r - k - \ell}{w - p} \binom{k + \ell}{p}}{\binom{t \cdot r}{w}}.$$

The time complexity T_{ISD} of the ISD algorithm can be obtained by using Equation (6).

Let $T_{\text{name}}^{(s,t)\text{-QCSD}}(n, \mathbf{w})$ and $T_{\text{name}}^{\text{SD}}(r, \mathbf{w})$ denote the time complexities of solving the (s, t) -QCSD($r, \mathbf{w}$) problem and the SD($sr, (s - t)r, w = \sum_{i=1}^s w_i$) problem, respectively, using the “name” ISD algorithm, where “name” refers to a specific algorithm from the list of known ISD algorithms, such as Prange, Dumer, MMT, BJMM, BM, and BO, etc. We summarize our attack as follows.

Theorem 4. *We have*

$$\frac{T_{\text{name}}^{(s,t)\text{-QCSD}}(r, \mathbf{w})}{T_{\text{name}}^{\text{SD}}(sr, (s-t)r, w)} = \frac{\prod_{i=1}^s \binom{r}{w_i}}{\binom{sr}{w}}, \text{ or equivalently,}$$

$$\log_2 T_{\text{name}}^{(s,t)\text{-QCSD}}(r, \mathbf{w}) = \log_2 T_{\text{name}}^{\text{SD}}(sr, (s-t)r, w) - \text{CombGap}(r, \mathbf{w}).$$

Empirically, the combinatorial gap is indeed non-negligible (see e.g. Table 6). We note that this also decreases the concrete bits of security levels of code-based cryptographic schemes such as HQC and BIKE, whose security relies on the hardness of $(s, 1)$ -QCSD problem.

By Theorem 4, the concrete security level of a PKE scheme based on the (s, t) -QCSD($r, \mathbf{w}$) problem can be computed as follows: first, determine the computational complexity of the underlying SD($sr, (s - t)r, w$) problem; then, take into account the speedup factor $\sqrt{r}$ contributed by the attacks exploiting the “Decoding-One-Out-of-Many” (DOOM) technique [20]; finally, deduct the combinatorial gap $\text{CombGap}(\mathbf{n}, \mathbf{w})$ introduced by the quasi-cyclic structure and unbalanced weights.

3.2.2 Quantum ISD Algorithms for QCSD Problems In quantum attacks against the SD problem, Bernstein’s Grover-based square-root speedup for Prange’s algorithm is the standard approach. Other quantizations of more advanced ISD algorithms, while asymptotically slightly better, introduce such large overheads that they hold no practical relevance for concrete security estimates. The computational complexity of the quantum version of the Prange algorithm is given below.

Proposition 6 (Quantum Prange algorithm [7, Section 5.1]). *Given a binary SD problem with parameters (n, k, w) , the quantum version of the Prange algorithm can find a solution to the problem with time complexity:*

$$Q T_{\text{Prange}}^{\text{SD}(n, k, w)} = \begin{cases} T/\sqrt{P} & \text{unlimited quantum circuit depth,} \\ T^2/(P \cdot 2^{\text{max-depth}}) & \text{quantum circuit depth} \leq \text{max-depth.} \end{cases}$$

Here, $\text{max-depth} \in \{40, 64, 96\}$, $P = \binom{n-k}{w} / \binom{n}{w}$, $T = (n-k)\omega$, and $\omega \in [2, 3]$ is the matrix multiplication constant.

Similar to the classical approach, we obtain the quantum complexity of (s, t) -QCSD($r, \mathbf{w}$) problem with unbalanced errors as follows, which is denoted $\text{QT}_{\text{Prange}}^{(s,t)\text{-QCSD}(r,\mathbf{w})}$.

Theorem 5. *We have that for quantum circuits with unlimited depth,*

$$\frac{\text{QT}_{\text{Prange}}^{(s,t)\text{-QCSD}(r,\mathbf{w})}}{\text{QT}_{\text{Prange}}^{\text{SD}(sr,(s-t)r,w)}} = \sqrt{\frac{\prod_{i=1}^s \binom{r}{w_i}}{\binom{sr}{w}}}, \text{ or equivalently,}$$

$$\log_2 \text{QT}_{\text{Prange}}^{(s,t)\text{-QCSD}(r,\mathbf{w})} = \log_2 \text{T}_{\text{Prange}}^{\text{SD}(sr,(s-t)r,w)} - \text{CombGap}(r, \mathbf{w})/2.$$

While for quantum circuits with limited depth,

$$\frac{\text{QT}_{\text{Prange}}^{(s,t)\text{-QCSD}(r,\mathbf{w})}}{\text{QT}_{\text{Prange}}^{\text{SD}(sr,(s-t)r,w)}} = \frac{\prod_{i=1}^s \binom{r}{w_i}}{\binom{sr}{w}}, \text{ or equivalently,}$$

$$\log_2 \text{QT}_{\text{Prange}}^{(s,t)\text{-QCSD}(r,\mathbf{w})} = \log_2 \text{T}_{\text{Prange}}^{\text{SD}(sr,(s-t)r,w)} - \text{CombGap}(r, \mathbf{w}).$$

In concrete quantum cost estimates including those presented in Table 6, we use the syndrome decoding estimator [7]; in particular, we only consider quantum complexity under unlimited depth, and thus the deduction due to combinatorial gap is $\text{CombGap}(r, \mathbf{w})/2$.

3.3 Other Attacks

Moreover, there exist attacks that could benefit from low-degree factoring of the modulus polynomial $x^r - 1$ for $\mathcal{R}$. Once $x^r - 1$ has only irreducible factors of the form $(x - 1)$ and $x^{r-1} + x^{r-2} + \dots + x + 1$, these attacks become ineffective. This is indeed the case if r is prime and 2 generates the multiplicative group $(\mathbb{Z}/r\mathbb{Z})^*$. In our parameter selection, we enforce this criterion for r .

Finally, we consider multi-ciphertext attacks [11] and add a salt in generating the ciphertext in QUBE.KEM construction.

3.4 Our Attacks on HQC

Here we quickly demonstrate the effectiveness of our attacks by applying them to the HQC algorithm, a NIST standard post-quantum algorithm. We list its parameter sets in Table 8.

It turns out that the security of HQC can be reduced to two hard problems: $(2, 1)$ -QCSD($r, (w_x, w_y)$) and $(3, 1)$ -QCSD($r, (w_{r_1}, w_{r_2}, w_e)$) (corresponding to our PK and CT problems, respectively). Security estimates for HQC are provided in e.g. [7]; recently, several improvements have been proposed in [8]. We apply our attacks of Theorem 4 to HQC, and present the combinatorial gaps for the hard problems in Table 9, along with the original and adjusted security levels. It can be observed that all parameter sets fall below (marked in red) the desired security levels of 143, 207, and 272 bits, which are complexity estimates for attacking AES-128, -192, and -256.

Table 8: Parameter sets and DFR for HQC.

Instance	λ	r	$w_x = w_y$	$w_{r_1} = w_{r_2} = w_e$	n_1	n_2	DFR Level
HQC-1	128	17669	66	75	46	384	132.89
HQC-3	192	35851	100	114	56	640	193.86
HQC-5	256	57637	131	149	90	640	260.59

Table 9: The combinatorial gaps and adjusted security levels of HQC

Instance	HQC-1	HQC-3	HQC-5
PK Problem ISD	144.71	212.91	275.08
CT Problem ISD	145.99	214.61	276.20
PK Comb. Gap	3.85	4.15	4.34
CT Comb. Gap	8.08	8.69	9.08
PK Problem Adjusted	140.86	208.76	270.74
CT Problem Adjusted	137.91	205.91	267.12

4 Performance Evaluations

4.1 Implementation Notes

To accommodate the development and deployment environments of different platforms, we provide the following explanations of the methods, techniques, and considerations used in the our implementations.

Language and platform. Our implementation is written in C and is compatible with most Linux development environments. It uses standard CMakeLists for compilation and management. The following library FIPS202.c are required to provide SHA-3 hash family variants of QUBE (see Section 1.5).

Modular design. Functions such as finite field operations, concatenated code encoding/decoding, polynomial multiplication, key/ciphertext (un)packing, and hashing/randomness generation are implemented as independent modules to facilitate maintenance and drop-in replacement.

Finite field operations. For finite field operations, the main relevant header files include `ffi_field.h`, `ffi_elt.h`, and `ffi_vec.h`, which provide arithmetic operations over the finite field $\mathbb{F}_2$ and vector space $\mathbb{F}_2^n$ over it.

Optimized key/ciphertext representation. A seed representation is taken to compress key sizes; e.g., the public key (of QUBE.PKE) is represented as $(\rho_0, \mathbf{S})$ where ρ_0 is the seed for generating $\mathbf{H}$, and the private key as ρ_1 , the seed for generating $\mathbf{y}$. The actual key data can be reconstructed very fast from the seeds, hence it is worth reducing the size overhead.

Table 10: Performance (in 10^6 clock cycles) of the reference implementation version of QUBE, compared to HQC.

Instance	KeyGen	Encaps	Decaps
QUBE-128	8.21	12.39	16.54
QUBE-256	41.00	59.71	82.43
QUBE-384	81.70	118.85	164.11
QUBE-512	82.63	116.73	161.17
HQC-128	3.82	7.70	11.82
HQC-256	27.71	55.28	83.56

Table 11: Performance (in 10^6 clock cycles) of the optimized implementation version of QUBE.

Instance	KeyGen	Encaps	Decaps
QUBE-128	0.107	0.212	0.338
QUBE-256	0.384	0.762	1.144
QUBE-384	1.077	1.910	3.152
QUBE-512	1.947	3.532	5.720

4.2 Metrics and Results

The reference implementation version is implemented in standard C and does not yet incorporate side-channel countermeasures.

Experiments are tested on a laptop equipped with a 2.50 GHz Intel(R) Core(TM) i9-12900H processor, 20GB of RAM, and running the Ubuntu 22.04.4 operating system. The executables are compiled using GCC 11.4.1. To mitigate system instability, each operation was repeated 1000 times in a single thread, and the average number of clock cycles is calculated. Table 10 shows the performance of all methods of QUBE, with a comparison to HQC.

In addition, Table 11 shows the performance of all methods of the optimized implementation version of QUBE.

5 Feature Statements

To summarize, we highlight the following main features and uniqueness of QUBE.

Additional functionality. As is often the case for post-quantum KEM schemes, QUBE is bundled with an IND-CPA-secure PKE scheme QUBE.PKE. When the application scenario merely asks for CPA instead of CCA security, QUBE.PKE may be utilized to achieve better performance.

New techniques. In QUBE, we use a multi-block technique and unbalanced block-wise errors.

Qualitatively improved security. The multi-block technique leads to a qualitatively weaker algebraic structure. E.g., compared to the $(2, 1)$ -QCSD and $(3, 1)$ -QCSD problems which HQC depends on, the underlying hard problems for QUBE is $(3, 1)$ -QCSD and $(4, 2)$ -QCSD which has more blocks and thus weaker cyclic structures.

Quantitatively improved security. We consider the effect of a new concept called combinatorial gap (Definition 6) in attacking the quasi-cyclic structure. E.g., we show in Section 3.4 that this new concept affects HQC dramatically, lowering its security levels to below the desired security levels in the NIST standardization. In QUBE, in contrast, we take the combinatorial gaps into consideration from design, and ensure that the parameter sets of QUBE stay securely above the desired security levels, under both classical and quantum attacks that make full use of the combinatorial gaps.

Quantitatively optimized ciphertext sizes. QUBE prioritizes to optimize the size of ciphertexts. The use of a multi-block technique and unbalanced blockwise errors together allow a much broader parameter space, and eventually enable us to select parameters that drastically cut down the size of ciphertexts in QUBE. When compared to HQC, there is a 25%–30% decrease in the size of ciphertexts. Moreover, we also do not sacrifice the total size of public key plus ciphertext, where a decrease of 1.4%–5.6% compared to HQC can be found.

We re-emphasize that the size of ciphertexts is particularly important in cryptographic protocols, especially when using KEM in static-ephemeral key agreement or KEM-based authentication like KEMTLS [19] with a static certificate. In normal ephemeral key agreements like in TLS, IPsec/IKE, or SSH, the total size of public key plus ciphertext often dominates the communication cost, and hence our improvements also make meaningful influence.

Qualitatively rebalanced sizes of public keys and ciphertexts. As a result of trying to optimize the size of ciphertexts, QUBE manages to gain a roughly 1:1 balanced ratio of the sizes of public keys and ciphertexts (see Table 5); in contrast, HQC has a roughly 1:2 ratio. As mentioned above, this more balanced ratio benefits most of the application scenarios that depend on transmitting both the public key and the ciphertext, while also specializes for scenarios that only transmit the ciphertext. As far as we know, there does not exist a scenario that involves merely the public key and no ciphertext, so we regard it nonbeneficial to have a public key size substantially smaller than the ciphertext size.

References

1. Aguilar-Melchor, C., Aragon, N., Bettaieb, S., et al.: Hamming quasi-cyclic (HQC) (2022) 3, 5, 12, 17
2. Becker, A., Joux, A., May, A., Meurer, A.: Decoding random binary linear codes in $2^{n/20}$: How $1 + 1 = 0$ improves information set decoding. In: EUROCRYPT. Lecture Notes in Computer Science, vol. 7237, pp. 520–536. Springer (2012) 20

3. Berlekamp, E.R., McEliece, R.J., van Tilborg, H.C.A.: On the inherent intractability of certain coding problems (corresp.). *IEEE Trans. Inf. Theory* **24**(3), 384–386 (1978) 9
4. Both, L., May, A.: Decoding linear codes with high error rate and its impact for LPN security. In: *PQCrypto. Lecture Notes in Computer Science*, vol. 10786, pp. 25–46. Springer (2018) 20, 21
5. Dumer, L.: On minimum distance decoding of linear codes. In: *5th Joint Soviet-Swedish Int. Workshop Inform. Theory.*, pp. 50–52 (1991) 20
6. Esser, A.: Revisiting nearest-neighbor-based information set decoding. In: *IMACC. Lecture Notes in Computer Science*, vol. 14421, pp. 34–54. Springer (2023) 20
7. Esser, A., Bellini, E.: Syndrome decoding estimator. In: *Public Key Cryptography. Lecture Notes in Computer Science*, vol. 13177, pp. 112–141. Springer (2022) 22, 23
8. Esser, A., Zweyding, F.: New time-memory trade-offs for subset sum - improving ISD in theory and practice. In: *EUROCRYPT. Lecture Notes in Computer Science*, vol. 14008, pp. 360–390. Springer (2023) 23
9. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. In: *CRYPTO. Lecture Notes in Computer Science*, vol. 1666, pp. 537–554. Springer (1999) 3, 14
10. Gaborit, P., et al.: Hamming quasi-cyclic (HQC). https://pqc-hqc.org/doc/hqc_specifications_2025_08_22.pdf (2025) 5, 6, 15, 16
11. Glabush, L., Hövelmanns, K., Stebila, D.: On the multi-target security of post-quantum key encapsulation mechanisms. *IACR Cryptol. ePrint Arch.* p. 343 (2025) 23
12. Goldwasser, S., Micali, S.: Probabilistic encryption. *J. Comput. Syst. Sci.* **28**(2), 270–299 (1984) 16
13. Hofheinz, D., Hövelmanns, K., Kiltz, E.: A modular analysis of the Fujisaki-Okamoto transformation. In: *TCC.* pp. 341–371. *Lecture Notes in Computer Science*, Springer (2017) 7, 19
14. Jiang, H., Zhang, Z., Ma, Z.: Tighter security proofs for generic key encapsulation mechanism in the quantum random oracle model. *Cryptology ePrint Archive, Paper 2019/134* (2019) 19
15. Lee, P.J., Brickell, E.F.: An observation on the security of McEliece’s public-key cryptosystem. In: *EUROCRYPT. Lecture Notes in Computer Science*, vol. 330, pp. 275–280. Springer (1988) 20
16. May, A., Meurer, A., Thomae, E.: Decoding random linear codes in $\mathcal{O}(2^{0.054n})$. In: *ASIACRYPT. Lecture Notes in Computer Science*, vol. 7073, pp. 107–124. Springer (2011) 20
17. May, A., Ozerov, I.: On computing nearest neighbors with applications to decoding of binary linear codes. In: *EUROCRYPT. Lecture Notes in Computer Science*, vol. 9056, pp. 203–228. Springer (2015) 20
18. Prange, E.: The use of information sets in decoding cyclic codes. *IRE Trans. Inf. Theory* **8**(5), 5–9 (1962) 20
19. Schwabe, P., Stebila, D., Wiggers, T.: Post-quantum TLS without handshake signatures. In: *CCS.* pp. 1461–1480 (2020) 3, 26
20. Sendrier, N.: Decoding one out of many. In: *PQCrypto. Lecture Notes in Computer Science*, vol. 7071, pp. 51–67. Springer (2011) 22
21. Stern, J.: A method for finding codewords of small weight. In: *Coding Theory and Applications. Lecture Notes in Computer Science*, vol. 388, pp. 106–113. Springer (1988) 20